CPU Wordle(가제) 프로젝트 설명

CPU Wordle(가제)은 한글과 영어 단어를 맞히는 웹 게임이다. 사용자가 단어를 입력하면 각 글자가 정답과 얼마나 일치하는지 색깔로 표시한다. 이 힌트를 이용해 제한된 횟수 안에 정답을 찾는다. 문제 코드를 공유하면 친구도 같은 문제를 풀 수 있다.

1. 주요 기능과 게임 규칙

초록은 글자와 위치가 모두 일치한다는 뜻이고, 노랑은 글자는 있지만 위치가 다르다는 뜻이다. 회색은 해당 글자에 대응할 정답 글자가 남아 있지 않다는 뜻이다. 같은 글자가 반복되면 정답에 있는 개수만큼만 초록이나 노랑을 준다.

모드	기회	규칙
기본	6번	단어 하나를 맞힌다.
해커	7번	오답마다 무작위 한 칸의 판정을 가린다. 입력한 글자는 보이며, 정답을 입력한 줄은 가리지 않는다.
탐정	공통 7번	이름 후보 100개 중 두 이름을 찾는다. 한 번의 입력을 두 정답과 동시에 비교하며, 두 이름을 모두 맞혀야 승리한다.

기본·해커 모드의 길이는 5~7칸이며 탐정 모드는 5칸 고정이다. 영어는 알파벳 수, 한글은 분해한 자음·모음 수로 칸을 센다. 예를 들어 사과는 ㅅ, ㅏ, ㄱ, ㅗ, ㅏ로 나누어 5칸이 된다.

입력한 단어의 칸 수와 사전 등록 여부를 확인해 수정이 필요한 내용을 안내한다. 이때는 기회를 그대로 유지한다. 탐정 모드에서는 화면의 이름 후보 목록을 참고할 수 있다. 한글은 글자 조합이 끝난 뒤 제출하도록 처리한다.

2. 프로그램 구성

화면은 HTML·CSS·JavaScript로 만들고, 게임 규칙은 C로 작성했다. C 코드를 WebAssembly라는 브라우저 실행용 형태로 변환해 사용한다. 게임 계산은 브라우저에서 이루어지며 별도 게임 서버가 필요하지 않다. web/index.html 파일을 직접 열어 실행할 수 있다.

파일	역할
src/wordle.c	한글 분해, 글자 판정, 문제 코드 생성·해석, 모드별 규칙, 승패 계산
web/engine.js	JavaScript에서 C 함수를 호출하고, 입력과 결과를 변환한다. 단어·이름 후보도 찾는다.
web/app.js	설정과 입력 처리, 진행 기록 관리, 게임판과 키보드 표시
web/index.html / styles.css	화면 구조와 디자인
web/wordle_wasm.js / words.js	브라우저에서 실행할 C 코드와 단어 데이터 번들

입력 처리 순서: 입력 확인 → 글자를 칸 단위로 분해 → C에서 판정·승패 계산 → JavaScript에 결과 전달 → 화면 갱신. C는 계산을 맡고, 입력 기록과 현재 진행 상태는 JavaScript 객체에 보관한다.

3. 한글을 자음·모음으로 분해하는 코드

위치: src/wordle.c의 wd_split 함수. 함수는 특정 작업을 수행하도록 묶어 둔 코드다. 아래 코드는 한글 한 글자에서 초성·중성·종성을 찾아 결과 목록에 넣는 부분이다.

```
uint32_t s = c - 0xAC00;
uint32_t initial[2] = {INITIAL[s / 588], 0};
count = push(out, count, cap, initial);
if (count >= 0)
    count = push(out, count, cap, VOWEL[s % 588 / 28]);
if (count >= 0)
    count = push(out, count, cap, FINAL[s % 28]);
```

컴퓨터에는 완성형 한글이 일정한 순서의 숫자로 저장되어 있다. 이 순서를 계산하면 첫 자음, 모음, 받침을 구분할 수 있다. INITIAL, VOWEL, FINAL은 각각에 대응하는 글자를 찾는 표이고, push는 결과 목록에 글자를 추가한다.

이 프로젝트의 분해 규칙에서 ㅘ는 ㅗ+ㅏ, 겹받침 ㄳ은 ㄱ+ㅅ으로 나뉜다. ㅐ와 쌍자음은 한 칸이다. 이 처리를 통해 한글도 영어처럼 칸별로 정답과 비교할 수 있다.

4. 중복 글자를 판정하는 코드

위치: src/wordle.c의 wd_judge 함수. 판정은 두 단계다. 먼저 위치까지 맞는 글자를 확정하고, 그다음 남은 정답 글자에서 위치가 다른 글자를 찾는다. 아래는 첫 번째 단계다.

```
for (int i = 0; i < n; i++) {
    out[i] = guess[i] == answer[i] ? WD_CORRECT : WD_ABSENT;
    used[i] = out[i] == WD_CORRECT;
}
```

guess는 입력한 글자 목록, answer는 정답 글자 목록이다. 같은 위치의 글자가 같으면 WD_CORRECT(초록)로 표시한다. used는 그 정답 칸을 이미 사용했는지 기록한다. 두 번째 단계에서는 사용하지 않은 정답 칸에 같은 글자가 있을 때만 노랑을 준다.

예를 들어 정답이 apple이고 입력이 allee이면 결과는 초록·노랑·회색·회색·초록이다. 정답에는 l이 하나뿐이므로 첫 l에 노랑을 주고 두 번째 l에는 회색을 준다. 마지막 e는 초록으로 먼저 확정했기 때문에 앞의 e에는 회색을 준다. 이 결과를 통해 글자의 위치뿐 아니라 정답에 들어 있는 개수도 추리할 수 있다.

5. 해커 모드에서 판정을 가리는 코드

위치: src/wordle.c의 wd_play 함수. 기본 판정을 계산한 뒤, 해커 모드의 오답이면 한 칸의 결과를 가림 상태로 바꾼다.

```
if (mode == WD_HACKER && !same(answers, guess, n)) {
    int hidden = (int)(random * n);
    results[hidden < n ? hidden : n - 1] = WD_HIDDEN;
}
```

첫 줄은 해커 모드이면서 입력이 정답과 다른지 확인한다. 다음 줄에서 무작위 칸 번호를 계산하고, 마지막 줄에서 해당 칸을 WD_HIDDEN(가림)으로 지정한다. 화면에서는 이 값을 줄무늬 칸으로 표시한다. 기본 판정 코드를 공유하면서 모드별 규칙을 추가한 부분이다.

6. 문제 코드 공유 방식

문제 코드는 모드, 언어, 단어 ID로 구성된다. 예를 들어 CW2-EN-F123CC0B는 기본 모드의 영어 문제이며, F123CC0B는 apple을 가리키는 ID다. 해커 모드는 CWH2, 탐정 모드는 CWD2로 시작하고 탐정 코드에는 두 이름의 ID가 들어간다.

단어 ID는 단어를 일정한 계산 규칙으로 숫자로 바꾼 값이다. 이런 계산을 해시라고 한다. 코드를 받은 브라우저는 형식을 확인한 뒤, 자신의 단어 목록에서 해당 ID의 단어를 찾는다. 사전의 정렬 순서가 바뀌어도 단어 자체에서 ID를 계산하므로 같은 정답을 찾을 수 있다.

화면에서 문제 코드를 복사해 친구에게 보내면, 친구는 코드 입력란에 붙여 넣어 같은 문제를 시작할 수 있다. 각자 추측한 단어와 풀이 과정을 비교하며 즐길 수 있다. 해커 모드에서는 같은 정답이라도 가려지는 칸이 달라져 서로 다른 단서를 얻게 된다.

7. 실행과 활용

web/index.html을 브라우저에서 열고 모드, 언어, 길이를 선택하면 바로 플레이할 수 있다. 다른 문제 버튼으로 같은 설정의 새 문제를 고르거나, 설정을 바꾸어 새로운 게임을 시작할 수 있다.

입력 중인 단어는 자음·모음 또는 알파벳으로 나뉘어 미리 표시된다. 키보드에는 지금까지 얻은 글자별 힌트가 모인다. 탐정 모드의 키보드는 왼쪽 절반에 인물 A, 오른쪽 절반에 인물 B의 판정을 표시하므로 두 정답의 단서를 함께 볼 수 있다.

참고하면 좋은 자료

README.md: 실행 방법, 모드별 규칙, 문제 코드 사용법을 볼 수 있다.

CODE_EXPLANATION.md: 주요 함수의 위치와 동작 원리를 코드 예시와 함께 설명한다.

src/wordle.c: 한글 분해, 중복 글자 판정, 모드별 진행 등 게임 규칙의 실제 구현을 볼 수 있다. 이 문서의 코드 세 부분도 여기에서 발췌했다.

web/engine.js와 web/app.js: C 계산 결과가 입력 처리와 게임 화면으로 이어지는 과정을 살펴볼 수 있다.

tests/test_engine.c와 tests/test_local.cjs: 다양한 입력에 어떤 결과를 기대하는지 구체적인 사례를 볼 수 있다.
scripts/build_wasm.py는 C를 브라우저용 파일로 변환하는 과정을 참고할 때 유용하다.